National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing

CourseEE-353-Computer Networks

Class: BESE-16AB

Project

Instructor: Ali Hassan

Lab Engineer: Rafia Ahmad

General Guidelines

- This is a **group project**. Each group may have a maximum of **two (2) students**.
- Write the **Name, Section, and CMS ID** of all group members on the first page of every submission.
- The project should address a **real-world problem**.
- The project must identify an **existing algorithm, protocol, method, or technique** related to the selected problem and propose a meaningful **improvement, modification, optimization, or extension**.
- The project must include implementation, experimentation, and performance evaluation.
- All figures, screenshots, graphs, tables, and outputs must be clearly labeled and explained.
- All submitted work must be original. **Plagiarism, copied work, or unauthorized sharing of project work will result in zero marks for both the sharer and the recipient.**
- Submit all required materials through the **LMS before the announced deadline**.
- Students are encouraged to use appropriate tools such as **Python, MATLAB, NS-3, Mininet, Wireshark, OMNeT++, or other relevant simulation/implementation platforms**, depending on the selected problem.

Important: Each project stage must be submitted by its announced deadline. **No extensions will be granted.** A penalty of **1 mark per day** will be deducted for late submissions. Students are strongly advised to submit their work before the deadline to avoid technical or LMS-related issues.

Project Objective

The objective of the project is to give students practical experience in solving a **real-world problem using an existing algorithm, protocol, or computational technique and improving its performance**. A successful project should follow this basic research direction:

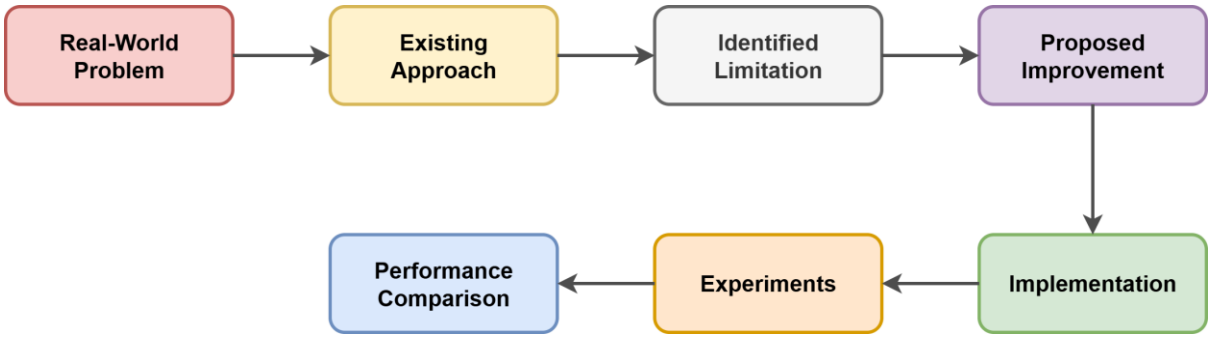


Figure 1: Overview of the successfully developed project pipeline.

The project should demonstrate that students can understand an existing solution, identify its limitations, develop an improved approach, and experimentally evaluate whether the proposed solution provides measurable benefits.

Four Project Stages:

Stage	Deliverable	Points
Stage 1	Project Proposal	**20**
Stage 2	Milestone Report	**20**
Stage 3	Final Report, Code & Demo Video	**40**
Stage 4	Viva Presentation	**20**
	Total	**100**

Stage 1: Project Proposal (20 Points)

Each team must submit a **two-page project proposal** containing:

- **Project Title**
- **Team Members**
- **Problem Description** — clearly explain the real-world problem being addressed.
- **Proposed Solution** — describe the basic idea of your proposed improvement.
- **Existing Approach** — identify the existing algorithm, protocol, model, or technique that you intend to improve.
- **Literature Review** — review relevant papers from approximately the **last 3–5 years**.
- **Literature Comparison Table** — compare multiple existing algorithms/approaches within the same problem domain.

Here is an example of a project comparison table. Note that the parameter fields should be changed to fit your problem.

Paper	Year	Problem	Existing Algorithm	Dataset/Environment	Key Metric	Main Limitation

The proposal should also specify:

- Expected **outcomes**
- Planned **experiments**
- Evaluation **metrics**
- Required **software/tools/resources**

The proposal should be approximately **two pages** and should clearly establish **what problem you are solving, what already exists, what limitation you identified, and what you plan to improve**.

Stage 2: Milestone Report (20 Points)

Each team must submit a **1–2 page milestone report** demonstrating meaningful progress toward the final project.

The report should include:

- **Project progress overview**
- Refined problem statement
- Defined system/project features
- Initial implementation
- At least **one existing/baseline algorithm successfully implemented and tested**
- Initial experimental results
- Graphs, tables, or visualizations
- Preliminary comparison/results
- Description of the proposed improvement
- Link/files for the current **source code**

At this stage, the project does not need to be complete. However, the team must demonstrate that the project is **actually being implemented and tested**, rather than remaining only a conceptual idea.

Stage 3: Final Report, Code & Demo Video (40 Points)

Final Research Report

Submit a **conference-style research paper** containing, at minimum:

1. **Abstract**
2. **Introduction**
3. **Problem Statement and Motivation**
4. **Related Work**
5. **Existing Approach**
6. **Proposed Methodology**
7. **Implementation / Experimental Setup**
8. **Results and Analysis**
9. **Comparison with Existing Approach(es)**
10. **Conclusion**
11. **References**

The report should clearly answer:

What problem did you solve?

What already existed?

What was its limitation?

What did you improve?

How did you implement it?

How did you test it?

What improvement did your results demonstrate?

Well-Documented Code

Submit the complete project source code.

The code should be:

- Properly organized
- Commented where necessary
- Easy to understand
- Reproducible
- Accompanied by a **README/User Manual**

The **User Manual** should explain:

1. Required software/dependencies
2. Installation/setup procedure
3. Dataset or input requirements, if applicable
4. How to run the project
5. Important commands/options
6. Expected output
7. How to reproduce the experiments/results

A student or evaluator who has **not developed the project** should be able to follow the instructions and run the project.

Demo Video: Designed for a Layperson

Each team must submit a **clear MP4 demo video**.

The video should not simply show the final output or consist of a screen recording of students reading their report.

The goal is:

A person who has never seen your project before should be able to watch the video and understand what the project does and how to run it.

The video should include the following:

1. Problem Explanation

Briefly explain the real-world problem in **simple, non-technical language**.

2. Existing Approach

Explain what existing algorithm/protocol/method is being used and **what limitation you identified**.

3. Proposed Improvement

Explain clearly:

- What you changed
- Why you changed it
- How the proposed approach works

4. Code Explanation

Show the important parts of the code and explain **what the code is doing**.

Do not explain every line. Focus on the main components, such as:

- Input
- Main algorithm
- Proposed modification
- Processing/workflow
- Output

5. Step-by-Step Execution

Demonstrate exactly how to run the project:

Setup → Command/Run → Input → Processing → Output

6. Results

Show the generated:

- Graphs
- Tables
- Metrics
- Screenshots
- Performance comparison

and briefly explain what the results mean.

7. Final Comparison

Clearly demonstrate the difference between:

Existing Approach vs. Proposed Improvement

The video should make it easy for a viewer to understand **what was improved and how the results demonstrate that improvement**.

Stage 4: Viva Presentation (20 Points)

Each team will deliver a **5–10 minute presentation in the lab**, followed by Q&A.

The presentation should cover:

- Problem and motivation
- Existing approach
- Identified limitation
- Proposed improvement
- Implementation
- Experimental setup
- Key results
- Comparison with the baseline

- Conclusion

The team must also **demonstrate the working code/project** during the viva.

Both group members should understand the project and be prepared to answer questions about the:

- Problem
- Algorithm
- Code
- Experimental setup
- Results
- Proposed improvement

Recommended Project Structure

Students are encouraged to follow this simple workflow:

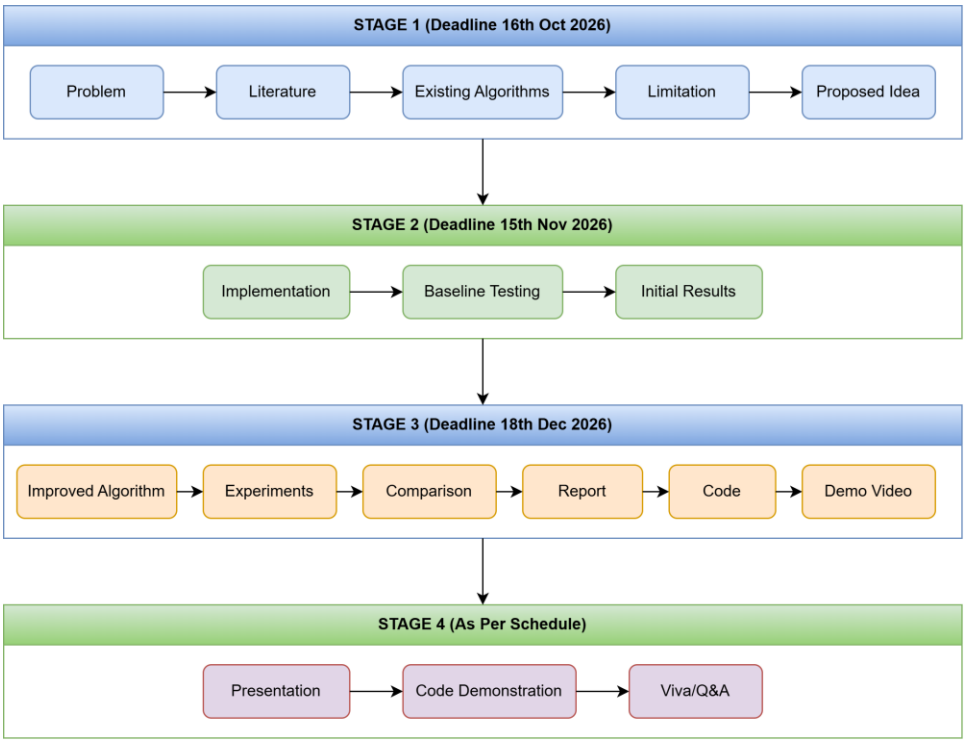


Figure 2: EE353 Course Project Development, Evaluation, and Submission Workflow

Important Advice

Do not wait until the final week to start implementation. A good project does not need to be unnecessarily complicated. **A clearly defined real-world problem, a properly implemented existing approach, a meaningful improvement, and convincing experimental evidence are more important than unnecessary complexity. Start early. Test frequently. Save your results. Document your work. And remember: sleep solves bugs.**